The Referee Is the Product:

Exploit-Tested Verification for LLM-Generated GPU Kernels,
and What It Falsified About Its Builders' Beliefs

Rohit Yelukati Mahendra

June 2026

Abstract

Our claim is narrow and methodological: in LLM-generated GPU kernel optimization, the *verifier* matters more than the proposer. The field's most publicized result was publicly corrected within days when reviewers found its kernels were exploiting the evaluation harness rather than accelerating the hardware; we read this as architectural, not anecdotal. The reward channel is usually treated as a passive metric when it has to be built and regression-tested as something an adversary will try to fool. Ouroboros is a verifier-grounded self-distillation loop built around an immutable referee that ships with negative controls for its own reward function: three working exploits (benchmark-shape special-casing, pointer-keyed output memoization, in-place input mutation) are committed as test cases and must be rejected for the system to be considered sound. Trained only against this referee, an open 27B model (Qwen3.6-27B with LoRA) produced 69 reproducible compiler-beating Triton fusion kernels, plus 7 exploratory kernels measured under a separate protocol (a single-shot probe, not the five-run gate). The 69 *all* clear a five-run stability gate against `torch.compile` max-autotune, and each also holds a geometric-mean win across a shape grid whose 10.9% losing cells we report individually. A three-seed ablation on a 1B model finds the training-recipe arms indistinguishable within seed noise on familiar operators, with best-of-N search against the referee accounting for nearly all of the loop's value; learning matters instead on operators the model has *not* seen. Finally, the referee falsified 9 beliefs we had recorded in advance, among them our own diagnosis of the system's one weak regime. We release the harness, kernels, corpus, adapters, raw measurements, and the prediction record. We claim verified scheduling wins on bandwidth-bound fusion operators; we do *not* claim new algorithms or gains over tuned libraries such as cuBLAS or FlashAttention.

1 Introduction

In February 2025, a well-funded research lab announced an "AI CUDA Engineer" with claimed speedups of up to 150×; within roughly 48 hours, public reviewers found the generated kernels were reusing memory across evaluation runs (gaming the benchmark harness, not accelerating the hardware) and the headline numbers collapsed [1]. The episode is usually read as a story about hype. It is better read as an engineering failure with an engineering fix. The reward channel of an RL-for-code pipeline is an attack surface, and an attack surface can be hardened, regression-tested, and shipped with its known exploits as test cases.

This paper describes Ouroboros, a system built on that observation, and reports what it did over one extended run.

What this paper is, and is not. It is *not* a new GPU algorithm; *not* a "bigger model writes better kernels" result; *not* a benchmark-leaderboard entry. It is a paper about building the reward channel itself as an **adversarially tested product**: a referee shipped with its own working exploits as regression tests. Everything that follows, the kernels, the stability gates, the ablations, is *evidence that this methodology works*, not the contribution. The thesis the rest of the paper is organized to test is blunt: **the proposer is replaceable; the referee is the durable asset.**

1. **The harness is the product** (§3). An immutable referee compiles each candidate Triton kernel, checks elementwise correctness against a PyTorch reference across *adversarial* shapes, dtypes, and

magnitudes, and measures wall-clock with CUDA events under a set of measurement guards (§3.2). The referee also ships with negative controls for its own reward channel: three cheat kernels, each a working exploit against a naive version of the harness, that must be rejected for the self-test to pass. The full self-test is 36 cases (17 gold kernels must pass, 19 negative controls must fail) and is ALL GREEN on both an RTX 4090 and an H200.

2. **The loop** (§4): supervised bootstrap to a measured validity gate, then verifier-rewarded search with self-distillation on the model's own verified winners (reward = measured speedup; Eq. 2), with dedup by canonicalized AST hash and winner's-curse-corrected exit validation.

3. **Results** (§5): 76 model-era kernels; 69/69 reproducible discoveries against `torch.compile` max-autotune (the incumbent autotuner, the strongest compiler baseline); geomean wins across the entire shape grid with losses (89/816 cells) reported per-cell and concentrated in one explainable regime; parity-or-better against expert hand-written Triton under *two* comparison conditions including the library-as-shipped one; and a correctness-gated end-to-end transformer-block composition with its Amdahl ceiling stated.

4. **Ablations** (§6): on familiar operators the recipe's arms are indistinguishable within seed noise (three seeds, on a 1B model), with search against the referee accounting for most of the result. Learning matters on operators the model has not seen (Fig. 3).

5. **A record of contradicted predictions** (§7): 9 cases where the referee disagreed with a belief we had written down in advance, among them our own diagnosis of the weak regime, a hand-written kernel of ours that the model beat, and an earlier claim of ours that we retract here.

We do not claim novel algorithm discovery (§8). The wins are scheduling wins of roughly 10 to 100% on bandwidth-bound fusion operators, the part of the kernel surface practitioners still write by hand. What we contribute is the method, not the kernels: against a verifier hardened and regression-tested against its known exploits, an open model out-searches the compiler's own autotuner and repairs its own weak cases. Every number in the paper is regenerated from a harness output by a script that also checks the reports against each other; a number that cannot be regenerated this way does not appear. To keep the contribution legible: the claims are hardware-specific (characterized on an RTX 4090 and an H200), and reproducibility is reported as the spread across five fresh runs rather than as a formal confidence interval; we make no claim beyond the operators, shapes, and machines measured.

2 Related Work

LLMs writing GPU kernels. KernelBench [2] established the evaluation regime; entrants include Kevin-32B (multi-turn RL) [3], AutoTriton (RL for Triton) [4], CUDA-L1 (contrastive RL) [5], and the AI CUDA Engineer [1]. Our work differs in locus: we contribute the *verification system* and treat the model as a replaceable commodity, our pipeline ran unchanged across models from 1B to 27B parameters (Section 6). We also intentionally evaluate against the compiler's own autotuner rather than eager PyTorch, since eager baselines flatter fusion work.

Machine-discovered algorithms and superoptimization. AlphaTensor [6] and AlphaDev [7] demonstrated the prize; classical superoptimization (e.g., STOKE [8]) established search-with-verification. OUROBOROS is the commodity-model, cheap-oracle version of this recipe, with the oracle-hardening itself as the contribution.

Reward hacking and verifiable rewards. RL from verifiable rewards is now standard in frontier training [9, 10]; reward gaming is its known failure mode. Our proposal, ship the reward function with implemented exploits as regression tests ("negative controls for rewards"), is one we are not aware of being systematically practiced in the RL-for-code literature, despite being cheap and decisive.

Compilers. `torch.compile`/Inductor with `max-autotune` [11] is our baseline *and* the incumbent we measure against; Triton [12] is the target language; Liger-Kernel [13] provides the expert hand-written comparison set.

storage dtype	rtol	atol
fp32	2×10^{-4}	2×10^{-5}
fp16	2×10^{-2}	2×10^{-3}
bf16	3×10^{-2}	8×10^{-3}

Table 1: Derived tolerance envelopes (internals accumulate in fp32). Per-operator overrides exist only where derived and documented (scan; entropy).

3 The Referee

A candidate kernel is a Python source string defining `run(*inputs)` and its `@triton.jit` kernel(s). The referee (`harness.py`, ∼370 lines, subprocess-isolated with hard timeouts) is the only component that ever executes one, and returns a verdict that neither the proposer nor any trainer can influence:

$$\text{verdict}(k) \;=\; \Big(\textsc{correct}(k) \in \{0,1\},\;\; s_b(k) = \frac{\text{median}_{i \le N}\, t_b^{(i)}}{\text{median}_{i \le N}\, t_k^{(i)}} \Big) \tag{1}$$

where b ranges over baselines (eager; `torch.compile` default; `torch.compile` max-autotune) and timings are CUDA-event medians (N=100 in-search, N=50 on grids).

3.1 Adversarial correctness

CORRECT(k) requires `allclose` against the PyTorch reference on *every* case in a sweep designed to make plausible-but-wrong kernels fail deterministically: (i) fixed stress cases at large magnitude (×64) in fp16/bf16 with non-power-of-two row lengths (e.g. 37 × 4097), these kill the classic traps (softmax without max-subtraction, RMSNorm without `rsqrt`, dropped residuals); (ii) **the benchmark inputs themselves** (§3.3); and (iii) k randomized shape/dtype/magnitude draws. Tolerances are *derived*, not tuned: for storage dtype with unit roundoff ε, reductions over D elements accumulate $O(\sqrt{D}\,\varepsilon)$ relative error in fp32, giving the per-dtype envelopes (rtol, atol) of Table 1. Where the global envelope is the wrong yardstick the override is derived at the spec and documented: for a length-N prefix scan the error tracks the *running-path magnitude* $\sigma\sqrt{j}$, giving atol $\approx c\,\varepsilon_{\text{fp32}}\,\sigma_{\max}\sqrt{N_{\max}}$; for entropy-from-logits the *reference itself* carries the cancellation $H = \text{lse}(x) - \sum_i x_i p_i$, bounding attainable agreement for *any* correct kernel. One candidate operator (`l1norm`) was *excluded* on these grounds: its outputs sit below the fp16 atol, so an all-zeros kernel would pass, an unverifiable spec is not allowed to exist in the suite.

3.2 Measurement hygiene

Every guard below exists because its absence produced (or would produce) a specific, recorded false result: **(a)** CUDA events with device synchronization, wall-clock around an async launch is meaningless. **(b)** Warmup of *both* paths (25 iters), otherwise one times JIT/Inductor compilation. **(c)** Median-of-N, boost clocks drift. **(d)** Clone/setup *outside* the timed window, an early version cloned inputs inside it, adding a ∼134 MB memcpy to every measurement and pulling all ratios toward 1.0 (manufacturing "ties" with the compiler). **(e)** Winner's-curse correction: the search archives the *max* over noisy measurements, a biased estimator; published numbers come from a fresh exit validation, and headline claims from the 5× stability gate of §5. **(f)** An opt-in *rotating-buffer* mode cycles 4 input clone-sets inside the timed loop (for candidate and baselines alike) so nothing stays L2-resident, the cache-cold cross-check.

3.3 Negative controls for the reward channel

The part that is unusual: we wrote three working exploits against the naive harness and committed them as permanent test cases:

1. **Bench-shape special-casing.** The timing inputs are fixed and public; a kernel that returns garbage instantly at exactly that shape would have passed the v1 correctness sweep (whose random generator

could not produce the bench shape) and posted an arbitrarily large "speedup." *Countermeasure:* the bench inputs are part of the correctness sweep, at every grid cell.

2. **Pointer-keyed memoization.** A `run()` that caches its output by input `data_ptr` does no GPU work after warmup. *Countermeasure:* before every timed iteration (outside the event window) one input element is overwritten with a *large in-distribution* value, and the final timed output is `allclose`-verified against the reference on the live input state. The first version of this poke, copying a neighboring element, was itself defeated by the control (the perturbation moved softmax outputs by $\sim 10^{-4}$, inside fp16 tolerance); the control caught the guard's weakness before any model could.

3. **In-place input mutation.** Correct values written into the input tensor make the timed loop's identical-arguments assumption unsound. *Countermeasure:* the correctness phase compares inputs before/after and rejects mutators outright.

All three must be REJECTED, each by its intended guard, with its specific error — for the self-test to pass. They are, on both GPUs tested (36/36 on NVIDIA GeForce RTX 4090; 36/36 on NVIDIA H200). We recommend treating these as *unit tests for reward functions* in any system trained against measured rewards.

4 The Loop

4.1 Operator suite

101 operator specifications: 16 explicit ops (norms, softmax family, GLU family, RoPE variants, int8 dequant, fused cross-entropy), a generative fusion grammar [±residual] → {rms, layer}norm → epilogue over 19 epilogues (76 chain ops, each verified with two gold template variants and cross-epilogue negative controls), and regime/algorithm-class probes (§5.5). An operator enters the loop only after its gold seed passes the referee *and* its negative control is rejected.

4.2 Phase 1: supervised bootstrap to a measured gate

A cold model emits 0% valid Triton. We SFT a LoRA on a *structurally diverse*, harness-filtered corpus (teacher-written implementations × launch-knob variants; only verified survivors enter), and gate on measured validity: sampled at the RL temperatures, per-op valid-rate $\geq 80\%$ with > 1 distinct structure (the recorded run reached 100%). Knob-twiddled copies of a few seeds teach "memorize N programs"; structural diversity is what teaches "write Triton."

4.3 Phase 2: verifier-rewarded search with self-distillation

Each round, for operator c: sample a group $\{k_i\}_{i=1}^{G}$ from $\pi_\theta(\cdot \mid c, \text{feedback})$, evaluate each through the referee (deduplicated by canonicalized-AST hash so a cosmetic rewrite is never re-measured), and assign

$$R(k) = \begin{cases} 1 + s_{\text{compile}}(k) & \text{CORRECT}(k) \\ 0 & \text{incorrect} \\ -0.2 & \text{runtime fail / timeout / crash} \\ -0.5 & \text{compile fail} \end{cases} \tag{2}$$

(correctness first, then speed, one scalar). The update combines a group-relative policy gradient [9] with self-distillation on the round's fastest verified kernel $k^\star$:

$$\mathcal{L} \;=\; \frac{1}{G}\sum_{i=1}^{G}\Big(-A_i \,\log \pi_\theta(k_i \mid c) \;+\; \beta \,\big|\log \pi_\theta(k_i|c) - \log \pi_{\text{ref}}(k_i|c)\big|\Big) \;-\; \log \pi_\theta(k^\star \mid c), \qquad A_i = \frac{R_i - \bar{R}}{\sigma_R} \tag{3}$$

where the second term is a *drift penalty* to a frozen reference, honestly labeled: it is an absolute sequence-log-probability difference, a crude one-sample KL proxy, and the production runs disable it (β=0) without

observed collapse. Stated plainly: with β=0, and with the policy-gradient term indistinguishable from zero effect in the multi-seed ablation (§6), the production update *reduces to rejection-sampled distillation on per-round champions*, best-of-G against the referee, then imitation of the verified winner. We keep Eq. 3 because it is what the code implements; the reduction is the honest summary of what the data say the update does, and it sharpens the thesis rather than weakening it: a sampler plus the referee is nearly the whole system. Prompts carry the referee's structured error feedback from the previous round (the fix-then-retry reflex). The archive keeps one champion per operator; exit validation re-measures every champion fresh with the `torch.compile` max-autotune baseline.

5 Results

All numbers in this section are emitted by the referee and compiled into this PDF by `make_numbers.py`, which also enforces cross-report consistency (12/12 checks at build time). Hardware: NVIDIA H200; torch 2.10; Triton 3.6; model Qwen3.6-27B (LoRA rank 128) unless stated.

5.1 The product: 69 reproducible discoveries (of 76 kernels)

What the two counts mean. The model authored 76 kernels in total. 69 of them are operators we carry through the five-run stability gate against `torch.compile` max-autotune below; the other 7 come from a later exploratory "invention" run (§5.5), measured as a single-shot probe rather than through the five-run stability gate, and so are not folded into the reproducibility numbers (they do beat `torch.compile` max-autotune; they are simply not re-run five times). Every reproducibility claim in this paper concerns the 69.

Family structure, stated up front. 57 of the 69 gated operators are instances of the one fusion grammar ([±residual] → {rms, layer}norm → epilogue); the remaining 12 span the softmax, GLU, RoPE, Gemma-RMSNorm, and cross-entropy families. A reader who counts schedule families rather than operators should discount "69 kernels" accordingly. The grammar exists to grade difficulty under controlled variation across epilogues, not to inflate the count; what keeps the count honest is that every operator clears the 5× stability gate and the shape-grid geomean *individually*, and the epilogues are not interchangeable (the softplus/mish family required an overflow-guard idiom the others never elicit; Fig. 3).

Stability gate. Every archived kernel re-benchmarked 5× fresh (full referee, anti-gaming guards live, `torch.compile` max-autotune recompiled each run). Discovery bar: $\bar{s} - \sigma > 1$, where $\bar{s}$ is the mean speedup and σ is the population standard deviation across the five fresh runs. We call σ the run-to-run *spread* throughout the paper. Result: **69/69 discoveries; zero failures; zero marginals** (Fig. 2). Mean of per-op means 1.299×; largest run-to-run spread $\sigma = 0.052$; weakest `add_layernorm_gelu_erf` at $1.113 \times \pm 0.015$; strongest `cross_entropy` at $2.038 \times \pm 0.031$ (model-authored).

Shape generality. Every kernel was re-benchmarked across a $(M, N) \times \{\text{fp16}, \text{bf16}\}$ grid (matrix family: 1024×4096 … 256×16384; rope family over (M, D)), with `torch.compile` max-autotune *recompiled per cell* and the anti-gaming guards active in every cell — 816 cells total. **All 69 kernels hold geomean > 1** (overall 1.494× on the 32 v1-era kernels, 1.480× on the 37 discovery-run kernels), all survive the cache-cold rotated check (69/69), and 89 cells (10.9%) are *losses, reported individually* (Fig. 1). One measurement caveat: each grid cell is a single measurement session (a median over 50 timed iterations), so a cell at 0.97 or 1.03 is a tie, not a loss or a win. Re-read with a ±3% grey band, the scale of the stability gate's run-to-run spreads, the grid gives 711 wins, 23 ties, and 82 losses; we report both readings so that neither marginal wins nor marginal losses are over-counted. The losses have structure under either reading: 76 of 89 strict losses (70 of 82 banded) concentrate at 16384 × 2048, many short rows, across two independently trained kernel generations. §5.5 returns to this regime.

5.2 Against expert hand-written Triton: two conditions

Comparisons against Liger-Kernel, Unsloth, and the Triton tutorials run through the same referee under two conditions: *fixed-schedule* (raw extracted `@triton.jit` kernels under our wrappers; schedule pinned) and *library-API* (Liger called via its own public `Function` interface, exactly as shipped). Ours is faster on swiglu/rmsnorm/relu2/geglu in both conditions; softmax and layernorm are ties-within-noise against the

best fixed-schedule expert. Two corrections. First, we retract our own earlier claim that 5 of 11 expert kernels were non-robust to odd shapes: under the library-API condition all 5 Liger recipes pass our adversarial sweep, the failures were artifacts of *our* extraction wrappers. Second, the forward-only cross-entropy comparison is *unfair to Liger* (their CE computes gradients in the forward by design) and is excluded from any headline. One correlation deserves stating plainly rather than leaving for a reviewer: our largest stability-gated win, `cross_entropy` at 2.038×, is exactly the operator this expert comparison excludes, and Inductor's cross-entropy lowering is a known weak point, so the `torch.compile` max-autotune baseline is softest precisely where our headline number is largest. The number is real, reproducible, and gated like every other; its margin should be read as a win over a weak compiler path, not over the strongest available implementation.

5.3 End-to-end composition

A Qwen-style MLP block (add_rmsnorm → gate/up GEMMs → swiglu → down GEMM) runs three ways on identical weights; output verified against eager (max err 2.9e-03) before timing. With trained kernels (NVIDIA H200): 1.143× vs eager and 0.993× vs whole-block compile-MA, the compile-MA comparison is *a tie, reported as a tie*: GEMMs are 81% of the block and identical across paths, bounding the achievable win (Amdahl), though the non-GEMM sub-path itself is 10.88× faster. (On an RTX 4090 with gold seeds the same block runs 1.085× / 1.302×, where the compiler is weaker.) The single-op wins survive composition; they cannot and do not claim block-level miracles.

5.4 Teaching 37 unseen operators in one run

Resuming the RL loop on 37 operators absent from all prior training (8 new epilogues × norm/residual variants, plus softcap-softmax, Gemma-RMSNorm, GLU, interleaved RoPE, fused cross-entropy), with the templated-mutation arm disabled (the model wrote every candidate): **37/37 validated correct and beat `torch.compile` max-autotune on fresh exit measurement** (1.16×–2.09×), 35/37 winning kernels model-authored, 617/888 sampled kernels verified (0.695), 101 archive lead-takes, explore-arm contribution **0**. The model beat its own strongest gold seed on fused cross-entropy (2.085× vs `torch.compile` max-autotune, model-authored). Learning is visible *inside* the run: the softplus/mish family, which requires an overflow-guard idiom (softplus$(x) = x$ for $x > 20$, else $\log(1+e^x)$), verified 3/1/1/1 per round on first encounter and 8/8/8/8 on the second pass, after self-distillation on the first pass's verified winners (Fig. 3).

5.5 The invention probe, and the repair of the loss regime

We then aimed the trained model at seven problems chosen to be *foreign*: `cumsum` (a prefix *scan*, carry propagation, a different parallel algorithm class from every reduction in the suite), `entropy` and `kl_div` (coupled double reductions over logits), and four operators pinned to the 16384×2048 loss regime identified by the shape grid. Pre-registered prediction (ours, in writing): the regime requires split-row/multi-row schedules.

Outcome: 7/7 validated and beat `torch.compile` max-autotune; 5/7 model-authored; validity on foreign problems 0.497 (vs ∼0.69 on familiar families, an honest difficulty gradient). Reading the kernels, not just the scores (Fig. 4):

- **The human diagnosis was wrong.** The winning kernels are plain *whole-row single-block* schedules; the losses were caused by looped templates tuned at N=4096, not by row-per-program parallelism. The worst published loss cell (0.69×, `add_layernorm_sigmoid`) became a 1.222× win; `layernorm_gelu` went 0.82× → 1.450×; softmax reached 1.803× in-regime.
- **`cumsum` is invention-lite, labeled precisely.** The model did not re-derive carry propagation; it recognized that a whole row fits one block and emitted a loop-free `tl.cumsum` kernel, beating our hand-written carry-loop gold by ∼47% (1.292× vs `torch.compile` max-autotune, 1.896× vs compile), correct across the full sweep. Primitive selection and schedule simplification, verified, not a new parallel algorithm.

arm	removes	valid-rate	beat MA	geomean vs MA
control	nothing (full loop)	1.000	8/8	1.338
no-feedback	referee errors in prompt	0.995	8/8	1.309
distill-only	policy-gradient term	0.948	8/8	**1.361**
no-learn	all weight updates	0.969	8/8	1.302

Table 2: Single-seed ablation on the 27B (familiar operators). The ordering here (distill-only nominally best) is *not* reproducible: the multi-seed replication of Table 3 flips it. We keep this table only to make that point.

arm	removes	valid-rate	beat-rate	geomean vs MA (3 seeds)
control	nothing (full loop)	0.972	100%	1.102 ± 0.028
no-feedback	referee errors in prompt	0.991	94%	1.090 ± 0.010
distill-only	policy-gradient term	0.944	83%	1.057 ± 0.030
no-learn	all weight updates	0.975	94%	1.079 ± 0.014

Table 3: Multi-seed ablation on MiniCPM5-1B, three seeds per arm. The arms do not separate beyond seed noise (max−min gap across arm geomeans 0.045, largest per-arm seed standard deviation 0.030; n=3 is too small for a formal test), and every arm beats max-autotune. The 1B model writing compiler-beating kernels at all is discussed in Section 8.

- **Where the model's competence stops:** on `entropy` and `kl_div` the model produced many verified kernels but never beat the hand-written two/three-pass golds (which themselves stand at 1.477× and 1.225× vs `torch.compile` max-autotune). Authorship there remains human.

The repair transfers. To test whether the whole-row family was four lucky per-op fixes or a learned skill, we pre-registered a prediction (this time it held) and resumed the loop from the invention adapter on the 10 worst-remaining loss-cell operators, pinned to 16384×2048. All **10/10 flipped to wins** (`torch.compile` max-autotune 1.20–1.44×, every kernel model-authored, explore-arm zero, validity 0.713). On this evidence the system's main characterized weakness, the short-row regime that accounts for the large majority of the shape grid's loss cells, is repaired for the chain family on the shapes and machines we tested. We read this as a generalization signal rather than a closed case: the fix is a schedule the model carries to operators it had not seen, not a memorized output, but it is established on the short-row family and hardware measured here, not proven in general.

6 Ablations: where does the value come from?

Four arms, identical except one ingredient (familiar ops, same SFT starting adapter, group 8): a single-seed pass on the 27B (Table 2) and a *three-seed* replication on a 1B model (Table 3, Fig. 5).

Multi-seed replication (and why the single-seed ordering is noise). A single seed per arm leaves the ordering open to the objection that n=1. We therefore repeated the ablation at three independent seeds per arm on a different and much smaller model, OpenBMB's MiniCPM5-1B, which reached the validity gate in 2 epochs (100% valid) and ran on a single RTX 4090. With three seeds per arm, n is too small for a formal hypothesis test and we do not dress one up: what the data show (Table 3, Fig. 5) is that the gaps between arms (max−min of the four arm geomeans, 0.045) are within seed noise (the largest per-arm standard deviation across seeds is 0.030). A second observation points the same way. Distill-only was nominally the best arm in the 27B single-seed run and is nominally the worst (1.057) in the 1B three-seed average; the same arm appears at both extremes, so the single-seed ordering was noise. The no-learn arm, which is best-of-N sampling from the frozen model with no weight updates, sits at 1.079, indistinguishable from control's 1.102.

The conclusion that survives the error bars is the weaker of the two. On familiar operators, best-of-N sampling against the referee captures essentially all of the full loop's geometric mean: search, not learning, dominates where the model is already competent, and this holds at both 1B and 27B. No single learning ingredient, whether the feedback channel, the policy-gradient term, or weight updates at all, separates

beyond seed noise in this regime. Learning matters on operators the model has not seen. The validity gap there (0.497 against roughly 0.69 on familiar families, Fig. 3) is the controlled contrast, and a 4.5-hour continue-SFT on new operators stalled and degraded the proven ones while RL against the verifier taught them without that interference. The new capabilities did not come from imitating a corpus.

We state the implication rather than hedge it. Where the model is already competent, the loop's value lives in *search against a trustworthy referee*, not in the training recipe, and this holds at both 1B and 27B. That is not a disappointing ablation; it is the thesis arriving on schedule. A proposer that contributes little beyond samples is a proposer you can swap for a cheaper one, which is precisely why the referee, and not the model, is the asset worth building and keeping.

7 The falsification ledger

We wrote our predictions down before running, which let the referee evaluate our judgement as well as the model's. Over the course of the project it contradicted 9 of our prior beliefs. The list includes our diagnosis of the loss regime (Section 5.5); our hand-written scan kernel, which the model beat by 47%; our earlier claim that several expert kernels were non-robust, which we retract in Section 5.2; the expectation that continue-SFT would teach new operators better than RL; the single-seed ablation ordering of Section 6; the adequacy of a small anti-memoization perturbation, which one of our own negative controls rejected before any model could exploit it; the verifiability of an operator (`l1norm`) whose outputs fall below the fp16 tolerance; an early benchmark that timed a memory copy and reported compiler ties; and the assumption that an all-green log without a durable artifact was sufficient evidence, which forced us to re-run a gate. One prediction held: that the loss-regime repair would transfer (10/10 in Section 5.5). We report this not for its own sake but because it is the practical consequence of the design. When the verifier is honest, the builders do not have to be right, and the record of where they were wrong is cheap to keep.

8 Limitations

(1) These are scheduling wins on bandwidth-bound fusion operators, with no claims against cuBLAS/FlashAttention-class kernels, and no novel algorithm classes (the scan result is primitive selection). (2) All speedups are vs `torch.compile` max-autotune on specific hardware (H200; spot-confirmed on RTX 4090); ratios are hardware-dependent even when reproducible. (2b) The pipeline was also run downward in scale, on OpenBMB's MiniCPM5-1B on a single RTX 4090. The 1B model reached the validity gate in 2 epochs (100% valid) and its kernels beat `torch.compile` max-autotune on every benchmarked operator (geometric mean 1.102×, Table 3). That a 1B model produces verified compiler-beating kernels on commodity hardware supports the central point: the proposer is replaceable across a wide range of scales, whereas the verifier is not. (3) The 27B ablation is single-seed; the 1B replication in Table 3 is three-seed. The asymmetry extends to the thesis itself: the claim that search dominates on *familiar* operators carries three-seed replication, while the more load-bearing claim, that learning matters on *unseen* operators, rests on one 37-operator run and one continue-SFT comparison, with no seed replication. The multi-seed discipline this paper argues for has so far been applied to the cheaper half of its own thesis. (4) KernelBench cross-evaluation is out-of-scope-by-construction for our specialist (foreign format, generalist operators); the recorded honest probe scored 0/1 and we declined prompt-bridging shortcuts that would manufacture a non-comparable number. (5) The loop requires a domain with a cheap, exploit-tested oracle and a measured quality signal; domains with judged rewards are explicitly outside the recipe, and the framework's failure modes (expensive oracles, adversarial environments, Goodhart on the metric) are discussed in the artifact.

9 Reproducibility

Everything regenerates from commands: the harness self-test (`harness.py`; 36 cases), the gates (`rebench_stability.py`, `rebench_shapes.py`), training (`modal_app.py` entrypoints), and this paper's every number (`paper/make_numbers.py`, which fails the build on cross-report inconsistency) and figure (`paper/make_figures.py`). The artifact includes the 76 kernels, adapters (1B and 27B), the verified corpus,

raw five-sample stability data, per-cell grid data, run transcripts where retrievable (with notes where logs are partial or truncated), and the falsification ledger.

10 Conclusion

We have treated the reward channel of an RL-for-code system the way security-critical code is treated, with its exploits written out, committed, and rejected on every build. Trained against that referee, an open model produced 69 reproducible compiler-beating kernels, plus 7 exploratory kernels under a separate protocol, repaired the one regime where it had been losing, and contradicted our own predictions repeatedly along the way. The proposer ranged from 1B to 27B parameters with no change to the pipeline; every proposer beat the autotuner, with margins that scaled with the model (geomean 1.102× at 1B, 1.299× at 27B). What did not depend on which model filled the slot was the soundness of the result. That depended on the referee. Where verification is cheap and the quality signal is measured rather than judged, this is the part of the system worth investing in, and the part that transfers to the next domain.

Reproducibility statement

Every numerical claim in this paper is regenerated from the harness's JSON outputs by a single script that also checks the reports against one another for consistency and aborts the build on any contradiction. Compute used H200 and L4 instances and a local RTX 4090.

References

[1] Sakana AI. *The AI CUDA Engineer*. Technical report, February 2025. https://sakana.ai/ai-cuda-engineer/. Headline speedups were corrected days later after community review found the evaluation harness was bypassed by a memory-reuse exploit.

[2] A. Ouyang, S. Guo, et al. *KernelBench: Can LLMs Write Efficient GPU Kernels?* arXiv:2502.10517, 2025.

[3] C. Baronio, P. Marsella, et al. *Kevin: Multi-Turn RL for Generating CUDA Kernels*. arXiv:2507.11948, 2025.

[4] S. Li et al. *AutoTriton: Automatic Triton Programming with Reinforcement Learning in LLMs*. arXiv:2507.05687, 2025.

[5] X. Li et al. *CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning*. arXiv:2507.14111, 2025.

[6] A. Fawzi et al. *Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning*. Nature **610**, 47–53, 2022.

[7] D. Mankowitz et al. *Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning*. Nature **618**, 257–263, 2023.

[8] E. Schkufza, R. Sharma, A. Aiken. *Stochastic Superoptimization*. ASPLOS, 2013.

[9] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024.

[10] N. Lambert et al. *Tülu 3: Pushing Frontiers in Open Language Model Post-Training*. arXiv:2411.15124, 2024.

[11] J. Ansel et al. *PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation*. ASPLOS, 2024.

[12] P. Tillet, H. T. Kung, D. Cox. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. MAPL, 2019.

[13] P.-L. Hsu et al. *Liger Kernel: Efficient Triton Kernels for LLM Training*. arXiv:2410.10989, 2024.

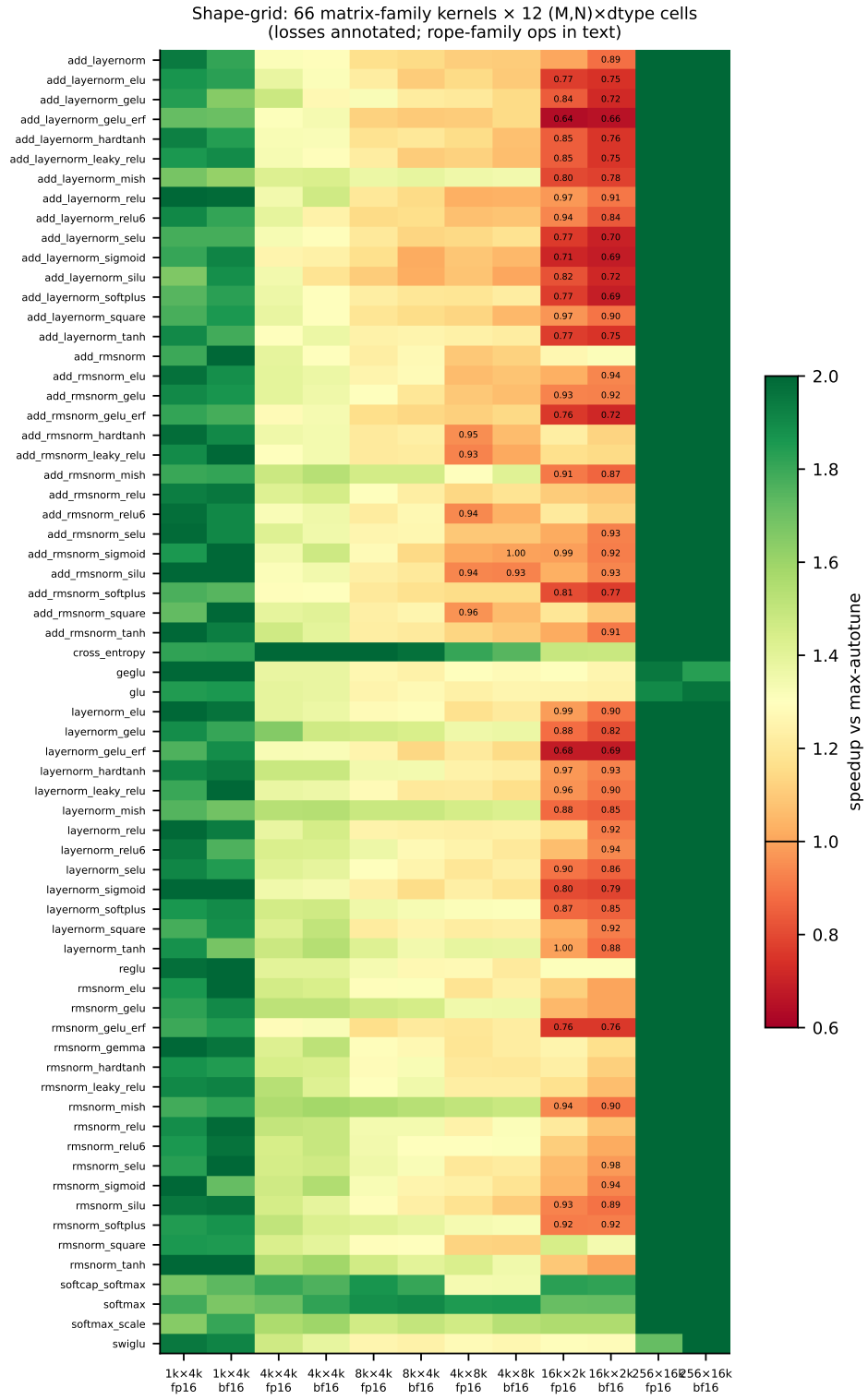

Figure 1: The shape grid. Green: beats `torch.compile` max-autotune (recompiled per cell); annotated red cells: losses, printed rather than hidden. The loss region is one regime, not noise.

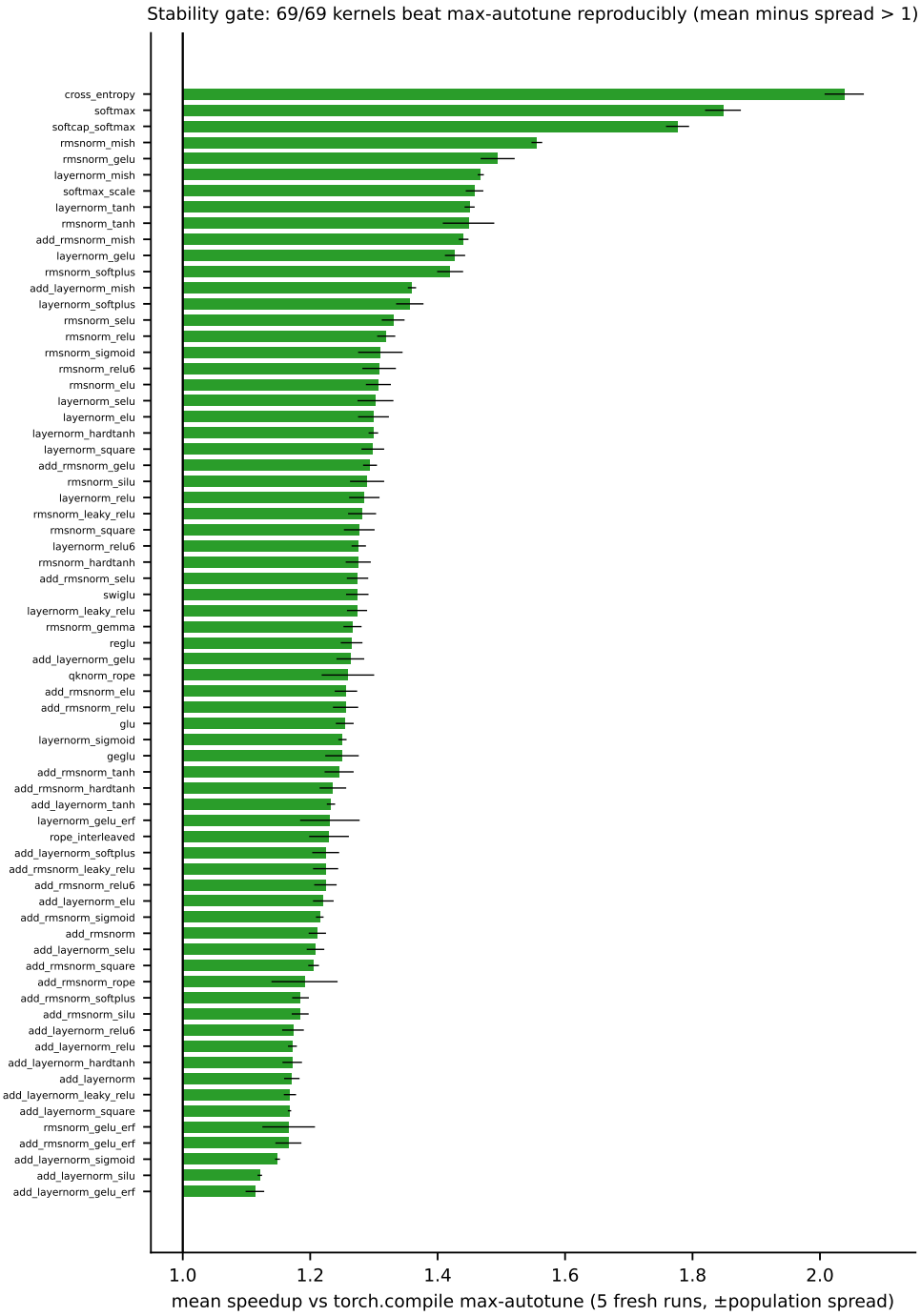

Figure 2: The 5× stability gate over all 69 kernels: mean ± spread vs `torch.compile` max-autotune. Raw five-sample data for every bar ships in the repository.

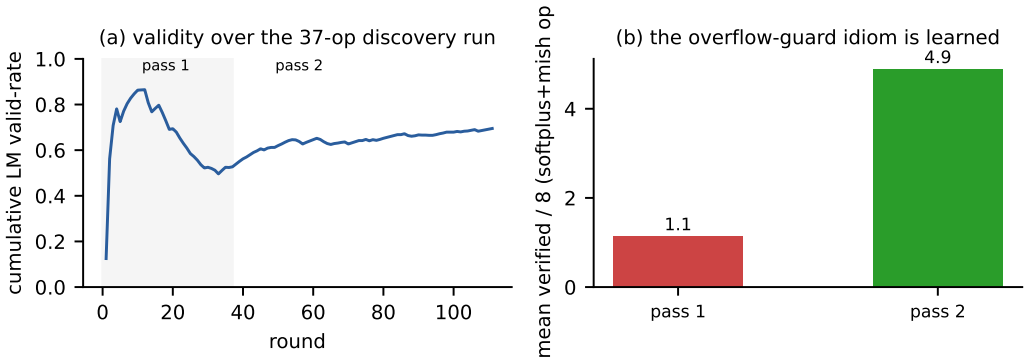

Figure 3: Learning where it matters: (a) cumulative validity over the 37-unseen-op run dips as harder epilogue families arrive, then recovers as distilled idioms generalize; (b) the overflow-guard idiom, quantified.

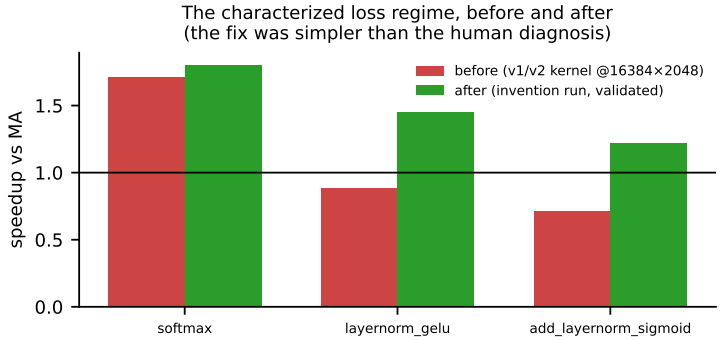

Figure 4: The characterized loss regime (16384×2048, fp16), before and after the invention run. The repair used a *simpler* schedule family than the authors predicted in writing.

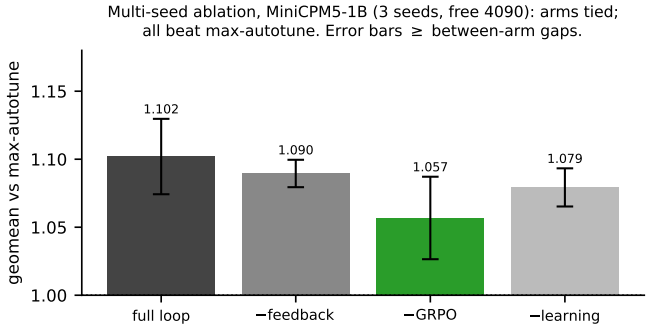

Figure 5: Multi-seed ablation (MiniCPM5-1B, 3 seeds). Error bars meet or exceed the gaps between arms: no learning ingredient separates on familiar operators.